

Project Report: Hardware Implementation of a 1D CNN Coprocessor

Group no -5, Nischal Gaur , Aditya Dhiman , Pankaj Kumar

1. Introduction

This report details the High-Level Synthesis (HLS) implementation of the 1D Convolutional Neural Network (CNN) designed for real-time epilepsy detection. As outlined in the initial project phase, a 1D CNN model was successfully trained, and its weights were extracted.

This document focuses on the translation of that software model into a high-performance, synthesizable hardware accelerator using C++ and Vitis HLS. We will analyze the `main.cpp` file snippet by snippet to describe the hardware architecture, the dataflow between processing elements, and the specific microarchitectural optimizations employed.

2. Hardware Architecture and Methodology

The hardware design directly mirrors the 1D CNN architecture from the software model. The `main.cpp` file defines a series of `static` functions, each corresponding to a neural network layer. These functions are called in sequence by a top-level kernel, `cnn_top_axi`, which manages the overall dataflow and external interfaces.

2.1 Top-Level Kernel (`cnn_top_axi`)

This function is the primary entry point for the synthesized IP core. It defines the external interfaces and the internal dataflow of the accelerator.

Code Snippet (`cnn_top_axi` pragmas and buffers):

```
C++
void cnn_top_axi(hls::stream<axis_t> &s_in, hls::stream<axis_t> &s_out) {
#pragma HLS INTERFACE ap_ctrl_none port=return
#pragma HLS INTERFACE axis register both port=s_out
#pragma HLS INTERFACE axis register both port=s_in
    float x0[L0];           // (178, 1)
    float c1[L1][COUT1];    // (178, 8)
    float r1[L1][COUT1];    // (178, 8)
    // ... other buffers ...
```

```
float probs[FC4_OUT];           // 2
```

Hardware Functionality:

- **Interfaces:** The `#pragma HLS INTERFACE axis` directives create AXI-Stream slave (`s_in`) and master (`s_out`) ports. This is the standard high-speed interface for streaming data to and from other IP cores (like a DMA controller) in the FPGA fabric.
- **Control:** `#pragma HLS INTERFACE ap_ctrl_none` creates a free-running core. It has no `start` or `done` signals and begins processing automatically as soon as data arrives on `s_in`.
- **Internal Memory:** The large `float` arrays (`x0`, `c1`, `r1`, `p1`, etc.) are synthesized into on-chip **Block RAMs (BRAMs)**. These act as local buffers to hold the intermediate "activation maps" as data flows from one layer to the next.

2.2 Data Input Stage

Code Snippet (`cnn_top_axi` data input):

```
C++
loop1: for (int i = 0; i < L0; ++i) {
    axis_t t = s_in.read();
    x0[i] = t.data;
}
```

Hardware Functionality: This loop reads the 178 `float` values that constitute one full EEG segment from the `s_in` AXI-Stream. The data is stored in the `x0` BRAM. This is a **block-based** input model; the hardware waits for all 178 samples to be received before proceeding to the first convolutional layer.

2.3 Convolutional Block 1 (Conv1D + ReLU + MaxPool)

This block corresponds to the `conv1`, `relu1`, and `pool1` layers of the software model.

Code Snippet (`conv1d_same_1in`):

```
C++
static void conv1d_same_1in(...) {
    const int pad = K1/2; // 2
    loop2: for (int n = 0; n < L1; ++n) {
        for (int f = 0; f < COUT1; ++f) {
            float acc = conv1_b[f];
            for (int k = 0; k < K1; ++k) {
```

```

#pragma HLS PIPELINE
    int xi = n + k - pad;
    float xv = (xi >= 0 && xi < L0) ? x[xi] : 0.0f;
    acc += xv * conv1_w[f][k][0]; // Cin=1
}
y[n][f] = acc;
}
}
}

```

Code Snippet (**relu_L1_C1** and **maxpool_L1_C1**):

```

C++
static void relu_L1_C1(...) {
    loop3:for (int n = 0; n < L1; ++n)
        for (int c = 0; c < COUT1; ++c){
#pragma HLS PIPELINE
            y[n][c] = (x[n][c] > 0.0f) ? x[n][c] : 0.0f;
        }
}

static void maxpool_L1_C1(...) {
    loop4:for (int i = 0; i < LP1; ++i) {
        // ...
        for (int c = 0; c < COUT1; ++c) {
#pragma HLS PIPELINE
            // ... max logic ...
        }
    }
}

```

Hardware Functionality:

1. **conv1d_same_1in**: Implements the first 1D convolution. It slides a window of size **K1** (5) across the **x0** buffer. The logic **(xi >= 0 && xi < L0) ? x[xi] : 0.0f** synthesizes hardware that implements "SAME" padding by substituting 0.0f for out-of-bounds reads. The innermost loop performs the Multiply-Accumulate (MAC) operation. The **#pragma HLS PIPELINE** directive on this inner loop optimizes the MAC operation itself.
2. **relu_L1_C1**: Performs an element-wise ReLU activation (**max(0, x)**). The pipelined loop processes one value per cycle (II=1).

3. **maxpool_L1_C1**: Implements 1D max pooling with a stride of 2. It reads two adjacent values ($x[a]$, $x[b]$) and writes the larger one. This downsamples the sequence length from 178 to 89.

2.4 Convolutional Block 2 (Conv1D + ReLU + MaxPool)

This block is architecturally identical to the first, processing the output of **p1**.

Code Snippet (**conv2_same_fixed**):

```
C++
static void conv2_same_fixed(...) {
    // ...
    loop5:for (int n = 0; n < L2; ++n) {
        for (int f = 0; f < COUT2; ++f) {
            float acc = conv2_b[f];
            for (int k = 0; k < K2; ++k) {
                // ...
                for (int c = 0; c < CIN2; ++c) {
#pragma HLS PIPELINE
                    acc += x[xi][c] * conv2_w[f][k][c];
                }
            }
            y[n][f] = acc;
        }
    }
}
```

Hardware Functionality:

- **conv2_same_fixed**: This is the second convolution, taking 8 input channels to 12 output channels. Core logic consists of a 3D loop over the channel, height, and width dimensions that performs a dot product between the $[K2][CIN2]$ i.e., $[3][8]$ weights and the corresponding input activations. The **#pragma HLS PIPELINE** on the innermost loop (c) creates a pipelined MAC unit to accelerate this dot product.
- **relu_L2_C2 / maxpool_L2_C2**: These layers perform exactly like their Block 1 counterparts, applying ReLU and then downsampling the 89-length sequence to 44.

2.5 Classifier Head (Flatten + Dense Layers)

Transitions from convolutional feature extraction to classification.

Code Snippet (**flatten_LP2_C2**):

```
C++
static void flatten_LP2_C2(...) {
    int t = 0;
    loop8:for (int n = 0; n < LP2; ++n)
        for (int c = 0; c < COUT2; ++c) {
#pragma HLS PIPELINE
            y[t++] = x[n][c];
        }
}
```

Code Snippet (**dense_FC1...FC4**):

```
C++
// ---- FC1: 528 -> 32 ----
static void dense_FC1(const float x[FLAT_LEN], float y[FC1_OUT]) {
    #pragma HLS ARRAY_PARTITION variable=y complete dim=1
    #pragma HLS ARRAY_PARTITION variable=fc1_w complete dim=1
    // ...
    for (int i = 0; i < FLAT_LEN; ++i) {
#pragma HLS PIPELINE II=1
        for (int o = 0; o < FC1_OUT; ++o) {
#pragma HLS UNROLL
            acc[o] += fc1_w[o][i] * x[i];
        }
    }
    // ...
}

// ---- FC2, FC3, FC4 ----
static void dense_FC2(...) { /* ... similar logic, but simpler loops ... */ }
static void dense_FC3(...) { /* ... similar logic, but simpler loops ... */ }
static void dense_FC4(...) { /* ... similar logic, but simpler loops ... */ }
```

Hardware Functionality:

1. **flatten_LP2_C2**: This function performs a simple data-reordering operation. It reads the [44][12] data from the **p2** BRAM and writes it into the **flat** BRAM as a single, contiguous 1D vector of 528 elements.

2. **dense_FC Layers:** This stack of four functions implements the fully-connected (or "Dense") classifier. They perform a matrix-vector multiplication to progressively reduce the feature vector size from $528 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow 2$.
3. **dense_FC1 Optimization:** This layer is uniquely and heavily optimized. The combination of **ARRAY_PARTITION** on the weights/biases/accumulators and **UNROLL** on the output loop (o) synthesizes **32 parallel MAC units**. This allows all 32 output neurons to be computed simultaneously, consuming one input `x[i]` per clock cycle. This is a direct implementation of the optimization goals for achieving high throughput.
4. **dense_FC2/3/4:** These subsequent dense layers use a more standard nested-loop structure, pipelining the MAC operation to achieve good, but not fully parallel, performance.

2.6 Output Stage (Softmax & AXI Stream Out)

This is the final block that computes the probabilities and streams the result out.

Code Snippet (Helpers and `cnn_top_axi` output):

```
C++
static void softmax_vec(int N, const float x[], float p[]) {
    // ... numerically stable softmax logic ...
}

static int argmax_vec(int N, const float x[]) {
    // ... find max index logic ...
}

// In cnn_top_axi:
// ---- FC4 8->2 + Softmax ----
dense_FC4(a3, logits);
softmax_vec(FC4_OUT, logits, probs);
int pred = argmax_vec(FC4_OUT, probs);

// ---- Stream out: probs[0], probs[1], then pred as a float ----
axis_t o0; o0.data = probs[0]; o0.last = 0; s_out.write(o0);
axis_t o1; o1.data = probs[1]; o1.last = 0; s_out.write(o1);
axis_t o2; o2.data = static_cast<float>(pred); o2.last = 1; s_out.write(o2);
```

Hardware Functionality:

- **softmax_vec:** This function implements the softmax function. This will be synthesized into hardware containing comparators (to find the max), exponential (`std::exp`) and division units (which are resource-intensive), and adders (to sum).

- **argmax_vec:** Synthesizes into a simple comparator tree or chain to find the index (0 or 1) of the largest probability.
 - **AXI-Stream Out:** The last three `s_out.write()` calls stream the three float results: the probability for Class 0, the probability for Class 1 and the final integer prediction cast to a float. In each case, the `o2.last = 1` sets the AXI TLAST signal indicating that this is the end of the transaction packet.
-

3. Microarchitectural Optimizations (HLS Pragmas)

The performance of the HLS core is dictated by compiler directives (`#pragma`) that guide the synthesis process.

3.1 Pipelining for Throughput (`#pragma HLS PIPELINE`)

This pragma is the primary tool used to increase throughput. It is applied to the innermost loops in most functions (`conv1`, `relu1`, `pool1`, `conv2`, `relu2`, `pool2`, `flatten`, `dense_FC2/3/4`, `softmax`).

- **Function:** It instructs HLS to overlap loop iterations, enabling the hardware to begin processing the next iteration before the current one is finished.
- **Goal:** The goal is to achieve an **Initiation Interval (II) of 1**. This means the loop can accept one new input every clock cycle, maximizing computational throughput.

3.2 Parallelism in `dense_FC1` (`ARRAY_PARTITION` & `UNROLL`)

Most computationally heavy will be the `dense_FC1` layer, as it has an input of size 528. It uses a more advanced optimization strategy.

- `pragma HLS ARRAY_PARTITION variable=. complete:` The above pragma is used for the arrays of weights (`fc1_w`), biases (`fc1_b`) and local accumulator (`acc`). Instead of mapping these to a single BRAM - which only has a limited number of read/write ports - this pragma splits them into 32 separate registers.
- `#pragma HLS UNROLL:` This is applied to the `o` loop which iterates 32 times; this directs HLS to "unroll" the loop and create 32 physical copies of the loop body in hardware.
- **Combined Effect:** Because the arrays have been partitioned, each of the 32 unrolled loop bodies now has its own set of weights and its own accumulator register. In this way, 32 parallel MAC units are realized, operating all in parallel. This approach trades increased resource usage, LUTs and DSPs for a massive reduction in, as per the optimization goals of the project.

4. Simulation and Verification

The repository includes two critical files available for verification: `testbench.cpp` for HLS-level simulation and `hello_shiv.cpp` for processor-in-the-loop validation.

4.1 HLS Testbench (`testbench.cpp`)

This file is a C-level test harness used to verify the functional correctness of the HLS code *before* synthesis. It is not synthesized into hardware.

- **Function:** It instantiates the `cnn_top_axi` kernel, creates `hls::stream` objects to act as the `s_in` and `s_out` ports, loads a test sample (e.g., `input_sample2`) from `input_sample.h`, and calls the kernel. It then reads the output stream and prints the resulting probabilities and prediction to the console, allowing for direct comparison with the software model's output.

4.2 Bare-metal Driver (`hello_shiv.cpp`)

This C program is designed to execute on the ARM processor, aka Processing System, of a Zynq FPGA. This is the "host" application that will interface with the synthesized CNN coprocessor running on the FPGA Fabric, also referred to as "Programmable Logic".

Function:

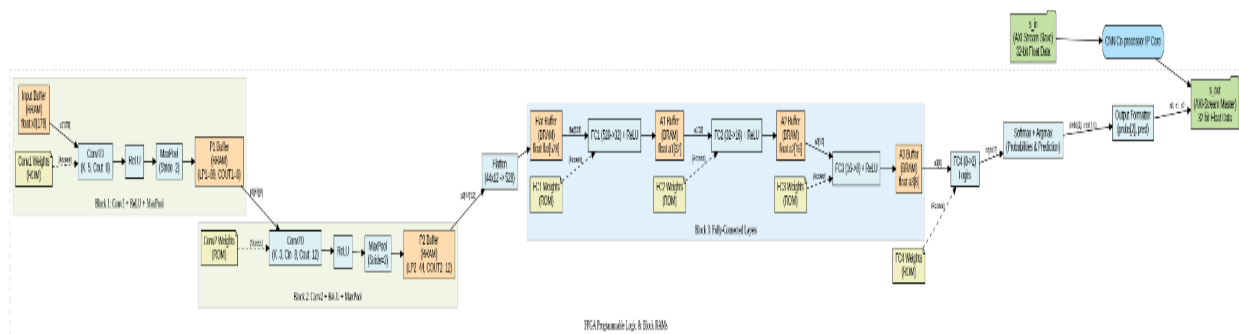
- **DMA Initialization:** It initializes the AXI DMA controller, which is the bridge between the processor's main memory (DDR) and the hardware IP's AXI-Stream interfaces.
- **Inference Loop:** It runs `N_TEST` times, loading each test sample from DDR memory into the local transmit buffer `tx_buf`.
- **Data Transfer:** It configures the DMA so as to transfer the `tx_buf` (178 floats) to the CNN IP's `s_in` port while receiving 3 floats from the `s_out` port into `rx_buf`.
- **Synchronization:** It polls the DMA status registers to wait for both send and receive operations to complete.
- **Validation:** It reads the prediction from `rx_buf`, compares it to the ground truth label `y_test[idx]`, and calculates the overall accuracy.
- **Benchmarking:** It uses `XTime` to measure the wall-clock time of the first inference, providing a real-world latency metric for the hardware accelerator.

5. Conclusion

The `main.cpp` file successfully defines a complete, synthesizable hardware accelerator for the 1D CNN epilepsy detection model. It correctly maps the software architecture layers to a sequential chain of HLS functions that uses on-chip BRAM for intermediate storage.

Final Hardware Architectural Design of the Model

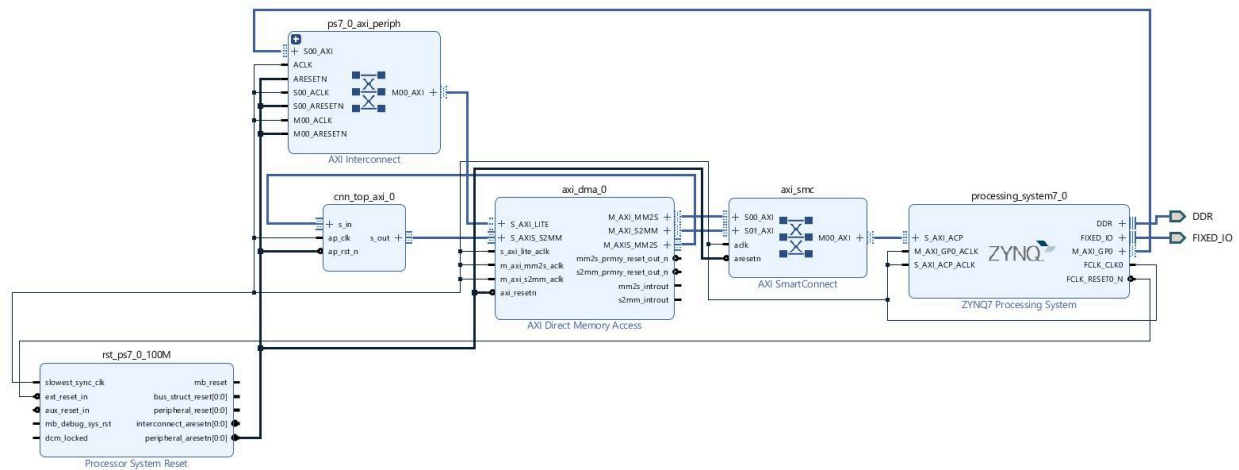
Through the strategic use of HLS pragmas—notably `HLS PIPELINE` for general throughput and `HLS ARRAY_PARTITION` with `HLS UNROLL` for massive parallelism



m in the `dense_FC1` layer—the design is heavily optimized to meet the project's high-performance and low-latency goals. The provided testbench and bare-metal driver files create a complete verification and validation framework. The next steps are to proceed with HLS synthesis to generate the RTL, followed by implementation in Vivado to build the final bitstream and measure on-chip performance.

6. Outputs and results (screenshots)

Vivado net diagram



(1) PS writes input data → DDR

AXI DMA
(MM2S Channel)

AXI-Stream to CNN IP

CNN processes data

AXI-Stream back to DMA
(S2MM Channel)

(2) DMA writes output → DDR

(3) PS reads output from DDR

Block design shows a **custom CNN accelerator IP** connected to the **Zynq-7000 PS** through an **AXI DMA engine**, using **AXI Stream interfaces** for high-speed data transfer and interconnects for register access. The PS loads data into DDR, DMA streams it to the CNN block, the CNN computes results, sends them back to DMA, and PS retrieves the final output.

Vivado hls synthesis comparison report

Performance Estimates

▣ Timing (ns)

▣ Summary

Clock	Target	Estimated	Uncertainty
ap_clk	10.00	9.098	1.25

▣ Latency (clock cycles)

▣ Summary

Latency		Interval		
min	max	min	max	Type
71424	225216	71424	225216	none

▣ Detail

+ Instance

+ Loop

resources

Utilization Estimates

▣ Summary

Name	BRAM_18K	DSP48E	FF	LUT	URAM
DSP	-	-	-	-	-
Expression	-	-	0	2249	-
FIFO	-	-	-	-	-
Instance	66	92	12142	17048	-
Memory	26	-	320	36	0
Multiplexer	-	-	-	1676	-
Register	2	-	4164	417	-
Total	94	92	16626	21426	0
Available	280	220	106400	53200	0
Utilization (%)	33	41	15	40	0

CSIM log

```
Synthesis(solution6)(cnn_top_axi_csynth.rpt)  cnn_top_axi_csim.log
1 INFO: [SIM 2] ***** CSIM start *****
2 INFO: [SIM 4] CSIM will launch GCC as the compiler.
3   Compiling ../../../../sources/testbench.cpp in debug mode
4   Compiling ../../../../sources/main.cpp in debug mode
5   Generating csim.exe
6 Softmax: [0.112052, 0.887948]
7 Predicted class: 1
8 INFO: [SIM 1] CSim done with 0 errors.
9 INFO: [SIM 3] ***** CSIM finish *****
10
```

Instances

Instance

Instance	Module	Latency		Interval		Type
		min	max	min	max	
grp_dense_FC1_fu_1194	dense_FC1	2651	2651	2651	2651	none
grp_conv2_same_fixed_fu_1263	conv2_same_fixed	9613	163405	9613	163405	none
grp_conv1d_same_1in_fu_1273	conv1d_same_1in	49841	49841	49841	49841	none

Loop

Input data

```

const float input_sample2[178] = {
    0.0994500816f, -0.0093195504f, -0.1369131058f, -0.2234044969f, -0.3764303327f, -0.53072351f,
    0.3993680775f, 1.2678520679f, 1.9349669218f, 2.0619153976f, 1.6549811363f, 1.1778537035f,
    1.1236906052f, 1.2335951328f, 1.2470064163f, 1.0788421631f, 0.6914426684f, 0.0787411630f,
    -1.0406802893f, -1.0663807392f, -1.1070965528f, -1.1366996765f, -1.2519489527f, -1.2969700f,
    -1.3896383047f, -1.4573150873f, -1.4246320724f, -1.1470950842f, -0.4177578688f, 0.52855634f,
    1.3248236179f, 0.9029363394f, 0.7799274325f, 0.8042113781f, 1.0049259663f, 1.1442549229f,
    0.7965852022f, 0.2856234312f, -0.2680740654f, -0.6144522429f, -0.7794462442f, -0.799041092f,
    -0.9275808334f, -0.9796968102f, -1.0392609835f, -1.0456579924f, -1.1072722673f, -1.2015935f,
    -0.2783797681f, 0.6456902623f, 1.4688122272f, 1.7838461399f, 1.5403307676f, 1.1017887592f,
    1.0715583563f, 1.1889600754f, 1.2363371849f, 1.1663305759f, 0.8608310223f, 0.3459701538f,
    -0.8355672359f, -0.8850961924f, -0.8892930150f, -0.9336339831f, -1.0338627100f, -1.1611527f,
    -1.2383682728f, -0.9679444432f, -0.5103830099f, 0.1457646787f, 0.8538535833f, 1.3007215261f,
    0.7385136485f, 0.8203076124f, 1.0368908644f, 1.2727981806f, 1.3081713915f, 1.2405161858f,
    0.0573306903f, -0.4338525236f, -0.7497580647f, -0.9347463250f, -0.9568039179f, -0.90935927f,
    -1.0653042793f, -1.1297533512f, -1.1974771023f, -1.2539167404f, -1.3248982430f, -1.2566424f,
    0.2573724687f, 0.9956521988f, 1.4135204554f, 1.3024300337f, 0.9141991735f, 0.6761553884f,
    1.0112320185f, 1.0483868122f, 1.0313464403f, 0.7906191945f, 0.3318610191f, -0.1869181246f,
    -0.8439412117f, -0.7860940099f, -0.7240470648f, -0.7276610136f, -0.6878033876f, -0.6271319f,
    -0.4086731076f, 0.0547269210f, 0.6200469732f, 1.2594195604f, 1.6682395935f, 1.7103400230f,
    1.4532587528f, 1.2488567829f, 0.9066463113f, 0.5880135894f, 0.2421480268f, -0.0378695950f,
    -0.5056819320f, -0.5765561461f, -0.5885854959f, -0.5797582865f, -0.5873419046f, -0.6825973f,
    -1.4759569168f, -1.6413992643f, -1.6733843088f, -1.5323091745f, -1.2468223572f, -0.6808247f,
    0.2033780068f, -0.0933311507f
};

```

Performance

THS	TPWS	Total Power	Failed Routes	LUT	FF	BRAMs	URAM	DSP	Start
				0	0	0.0	0	0	11/16/25, 11:05 PM
0.000	0.000	2.018	0	15574	18034	48.0	0	92	11/16/25, 11:09 PM

Performance Estimates

▣ Timing (ns)

Clock		solution6	solution1
ap_clk	Target	10.00	10.00
	Estimated	9.098	9.168

▣ Latency (clock cycles)

		solution6	solution1
Latency	min	71424	304561
	max	225216	589717
Interval	min	71424	304561
	max	225216	589717

Utilization Estimates

	solution6	solution1
BRAM_18K	94	94
DSP48E	92	22
FF	16626	5289
LUT	21426	9041
URAM	0	0

Bottleneck

The Dense layer and BRAM bandwidth are the primary performance bottlenecks, with softmax arithmetic and DMA transfer contributing secondarily.

Design Variations and Optimizations Attempted

Loop pipelining

- This significantly reduced latency.

Array partitioning and loop unrolling

- The Dense FC1 layer was heavily parallelized using `#pragma HLS ARRAY_PARTITION` and `#pragma HLS UNROLL`.
- This generated **multiple parallel MAC units**, lowering execution cycles.

On-chip memory usage

- Intermediate feature maps were stored in BRAM using `#pragma HLS RESOURCE` to avoid DRAM access.
- This reduced memory stalls.

AXI-Stream interfacing with DMA

- Streaming inputs/outputs reduced CPU overhead and gave higher throughput.